

Engineering and Product Justification

The AI Relationship Manager Agent was designed with the objective of balancing three often competing requirements: providing a high-quality conversational experience, maintaining factual accuracy, and ensuring compliance with financial advisory constraints. Throughout the project, several engineering and product decisions were made to achieve this balance, with each decision involving tradeoffs between simplicity, performance, reliability, and safety.

A key design decision was the adoption of a Retrieval-Augmented Generation (RAG) architecture in the early phases of development. Financial recommendations and market discussions often require access to domain-specific information that may not be present in the language model's training data. By integrating analyst reports and investment research documents into a vector database, the agent could provide responses grounded in relevant financial context. Over here the dataset we used for RAG was limited, so we did not perform any chunking and other things here, but as and when the data increases we will have to perform that as well. The primary advantage of this approach was improved factual coverage and explainability. However, testing revealed that RAG alone was insufficient for handling client-specific queries such as portfolio analysis, trade execution, or account-level calculations. This limitation motivated the transition to a tool-enabled architecture.

The second major design decision was the introduction of tool calling. Dedicated tools were developed for portfolio retrieval, trade-cost calculation, exchange surveillance checks, market-data retrieval, and trade confirmation workflows. This allowed the system to separate reasoning from data access. The language model became responsible for interpreting user intent and orchestrating workflows, while tools became the source of truth for financial information. The tradeoff was increased system complexity, as multiple APIs and validation layers had to be maintained, but the gain in accuracy and reliability significantly outweighed the added engineering effort. In a financial setting, deterministic tool outputs are considerably safer than relying solely on model-generated responses.

Another important product decision involved the implementation of a confirmation-based transaction workflow. Financial transactions are inherently high-risk actions, and accidental execution could have serious consequences. To address this, the agent was intentionally designed so that it cannot execute a trade immediately after a user request. Instead, it first performs safety checks, retrieves market information, calculates exact costs, and then presents a summary requiring explicit user confirmation through the keyword "CONFIRM". This introduces an additional interaction step, which may slightly reduce convenience, but substantially improves user control and reduces the risk of unintended transactions.

Safety and compliance considerations influenced several architectural choices. The system includes guardrails that validate tool usage, restrict unsupported actions, and prevent unsafe behavior. Requests related to derivatives and F&O trading are refused because the project

scope is limited to equity investments. Similarly, the agent does not provide guaranteed-return predictions and does not claim certainty regarding future market movements. These restrictions reduce the breadth of advice that can be offered but align the system with responsible financial-assistance principles. Additional safeguards were implemented to prevent unauthorized tool usage, repeated tool-call loops, and attempts to bypass the confirmation process.

Conversation memory was introduced to improve user experience during longer interactions. Without memory, the agent would lose context between turns and repeatedly ask for information that had already been provided. However, retaining every message indefinitely increases token usage and can negatively affect performance. To address this tradeoff, a memory-compression strategy was implemented in which older conversation history is summarized once a predefined threshold is reached. This preserves important context while controlling latency and token consumption.

From a deployment perspective, the system assumes the availability of reliable backend services for portfolio management, market-data retrieval, and order processing. The architecture separates the conversational agent from these services through API interfaces, making it easier to replace or extend individual components without affecting the overall system. While the current implementation is intended as a prototype, the design follows patterns commonly used in production AI systems, including tool abstraction, API-driven workflows, structured logging, and automated evaluation.

Overall, the final architecture reflects a deliberate tradeoff between flexibility and control. Rather than allowing the language model to answer every question independently, the system prioritizes grounded responses through retrieval, tool calling, and validation mechanisms. This approach increases engineering complexity but produces a more reliable, transparent, and trustworthy AI Relationship Manager, which is essential in a financial advisory environment.